EE 250L: Distributed Systems for the Internet of Things

Fall 2026 - Lab 2

TCP, UDP, Socket Programming, and ESP32-to-VM Communication

	Tuesday Lab	Thursday Lab
Assigned Date	9/8/26	9/10/26
Submission Due Date	9/14/26	9/16/26
Interview + Demo Due Date	9/15/26 (Before Lab Starts)	9/17/26 (Before Lab Starts)

Lab outcome: Compare TCP and UDP experimentally; implement a TCP client and server using Python sockets in the Ubuntu VM; and then replace the Python client with an ESP32-S3 TCP client written in C using ESP-IDF while keeping the same Python server running in the VM.

How this lab extends Lab 1

Lab 1 established two system roles: the Ubuntu VM as the Linux host/server environment and the ESP32-S3 as the embedded endpoint. In Lab 1 these systems were tested independently. In this lab they will communicate directly over Wi-Fi using TCP/IP.

Part	Systems used	Main goal
1	Ubuntu Linux VM	Observe TCP and UDP behavior, including under artificial packet loss.
2	Ubuntu Linux VM	Run a Python TCP server and Python TCP client on the VM and understand the socket API.
3	ESP32-S3 + Ubuntu Linux VM	Keep the same Python TCP server and replace the Python client with an ESP32-S3 client written in C.

Required hardware and software

- The Ubuntu Linux VM configured in Lab 1.
- ESP32-S3-DevKitC-1 and USB data cable configured in Lab 1.

- A 2.4 GHz Wi-Fi network or phone hotspot that can place the ESP32-S3 and the laptop/VM on a mutually reachable local network.
- Python 3 in the Ubuntu VM.
- Netcat and Linux traffic-control tools in the Ubuntu VM.
- ESP-IDF v6.0.2 on the laptop native operating system, as configured in Lab 1.
- Course-provided Lab 2 starter files. The source files will be distributed separately from this document on our Google Drive under Lab 2 here: https://drive.google.com/drive/folders/1KxnI7DXR12oGGMn2fUq8-Pkszb30Ze_x?usp=sharing .

Reminder: Do not submit Wi-Fi passwords or other credentials in source files, screenshots, Git commits, or public repositories.

Part 1: Evaluating TCP and UDP in the Linux VM

At the end of this section, you should be able to create simple TCP and UDP endpoints with netcat, identify the role of a port number, and experimentally compare how TCP and UDP behave when packet loss is introduced.

1. Install the networking tools

Open a terminal in your Ubuntu VM and run:

```
sudo apt update
sudo apt install -y netcat-openbsd iproute2
```

Verify that netcat is installed:

```
nc -h
# Press q or Ctrl+C if needed to return to the shell
```

The iproute2 package provides the tc command used below to emulate packet loss.

2. Review addresses and ports

A network application typically identifies a remote socket using an IP address and a port number. The IP address identifies a host/interface; the port number identifies a particular application endpoint on that host.

Term	Meaning in this lab
localhost / 127.0.0.1	The VM itself. Traffic sent to this address does not leave the VM.
Port	A 16-bit number used by TCP or UDP to identify an application endpoint. We will use unprivileged ports such as 10000 and 12345.
TCP	Connection-oriented byte-stream transport with reliability, ordering, and retransmission.

Term	Meaning in this lab
UDP	Connectionless datagram transport. The network does not retransmit a lost UDP datagram for the application.

3. Test a local TCP connection with netcat

Open two terminals in the Ubuntu VM.

Terminal 1 - TCP listener

```
nc -l 12345
```

The terminal will appear to wait for a connection. That is expected.

Terminal 2 - TCP client

```
echo "Hello, TCP!" | nc localhost 12345
```

Return to Terminal 1. You should see the message. Press Ctrl+C to stop the listener when finished.

4. Test UDP communication

Again use two terminals. In Terminal 1, start a UDP listener:

```
nc -u -l 10000
```

In Terminal 2, start a UDP client:

```
nc -u localhost 10000
```

Type the numbers 1 through 10 in Terminal 2, pressing Enter after each one. Verify that they appear in Terminal 1. Leave both terminals open for the next step.

5. Introduce 50% packet loss and observe UDP

Open a third terminal. The following command adds artificial packet loss to the VM loopback interface lo:

```
sudo tc qdisc add dev lo root netem loss 50%
```

In the UDP client terminal (Terminal 2), send the sequence 1 through 10 several more times. Observe which datagrams appear in the listener. Because the loss process is random, the exact messages lost will differ from run to run.

> **If the command reports that a qdisc already exists:** First remove an old rule with sudo tc qdisc del dev lo root, then add the loss rule again.

When finished with the UDP experiment, remove the loss rule:

```
sudo tc qdisc del dev lo root netem
```

6. Repeat the loss experiment using TCP

Stop the UDP processes. Start a TCP listener in Terminal 1:

```
nc -l 10000
```

Start a TCP client in Terminal 2:

```
nc localhost 10000
```

Type the numbers 1 through 10 and verify that they arrive. Then add 50% loss again in Terminal 3:

```
sudo tc qdisc add dev lo root netem loss 50%
```

Send the sequence again. TCP should attempt to recover from packet loss through retransmission. You may notice delay or temporary pauses rather than missing application data.

When finished, always remove the rule:

```
sudo tc qdisc del dev lo root netem
```

Questions for Part 1

1. How did introducing 50% packet loss affect the UDP messages you observed? Explain why.
2. How did introducing 50% packet loss affect the TCP exchange? Did the application data disappear, arrive later, or behave differently? Explain why.
3. Why is localhost useful for testing applications even though no packets need to traverse the physical Wi-Fi network?

Part 1 complete: You have demonstrated TCP and UDP communication in the VM and observed how the two transports react differently to induced packet loss.

Evidence: Include screenshots showing your TCP and UDP netcat tests and at least one loss experiment. Your screenshots should make clear which terminal is the sender and which is the receiver.

Part 2: Python TCP Client and Server in the Linux VM

In Part 1, netcat hid most of the socket-programming details. You will now use Python to work directly with the socket interface. Both programs initially run in the same Ubuntu VM so that you can focus on the programming model before introducing the ESP32-S3.

1. Obtain the Lab 2 starter files

Download the course-provided Lab 2 starter files into a working directory in your Ubuntu VM. The starter bundle will include at least:

- tcp_server.py - the TCP server used in Parts 2 and 3.
- tcp_client.py - the Python client used in Part 2.
- README.md (or equivalent response file) - where you will record answers to the lab questions.

For example, you may use:

```
mkdir -p ~/ee250/lab2
```

```
cd ~/ee250/lab2
# Place the course-provided Lab 2 files here
```

Source code: The full source code is distributed separately. This lab document describes what the programs must do and which concepts you must understand.

2. Understand the client/server sequence

The server waits for an incoming TCP connection. The client initiates the connection. Once connected, the client sends a message, the server prints the message and sends a reply, and the client prints the reply.

Python TCP server	Python TCP client
Create a socket	Create a socket
Bind the socket to a local port	Connect to the server IP address and port
Listen for incoming connections	Send an application message
Accept a connection	Receive the server reply
Receive the client message	Close the socket
Send a reply	
Close the connection/socket	

The important Python socket operations will include socket(), bind(), listen(), accept(), connect(), sendall()/send(), recv(), and close(). You should be able to explain the purpose of each operation used by the supplied programs.

Fill in the TODOs in tcp_client.py before proceeding to the next step

3. Run the Python server

In Terminal 1, change to your Lab 2 directory and start the server using the port specified by the starter code/instructions. For example:

```
cd ~/ee250/lab2
python3 tcp_server.py 5000
```

The exact command-line format will be finalized with the supplied code. The server should print that it is listening and wait for a client.

4. Run the Python client using localhost

In Terminal 2, run the Python client and direct it to localhost and the same server port. For example:

```
cd ~/ee250/lab2
python3 tcp_client.py 127.0.0.1 5000
```

Enter a message when prompted. The expected application behavior is:

- The client reads a line of input from you.
- The client sends that message to the TCP server.
- The server prints the received message.
- The server replies with the exact acknowledgment string defined by the starter code, such as “I got your message”.
- The client receives the acknowledgement and prints it.

Repeat with at least two different messages. Stop the server with Ctrl+C when finished.

5. Inspect the VM network address

Part 3 will require the ESP32-S3 to connect to the VM over the local network. Find the VM IPv4 address now:

```
hostname -I
ip addr
```

You may see more than one address. Identify the address associated with the VM interface connected to your local network. Addresses such as 10.x.x.x, 172.16-31.x.x, or 192.168.x.x are private IPv4 addresses and are normal.

Do not use 127.0.0.1 in Part 3: On the ESP32-S3, 127.0.0.1 would refer to the ESP32-S3 itself, not the Ubuntu VM.

Questions for Part 2

1. Which side calls listen()/accept(), and which side calls connect()? Why?
2. What two pieces of information does the client need in order to connect to the server?
3. What is the difference between 127.0.0.1 and the VM IPv4 address you found with hostname -I or ip addr?
4. The same TCP connection can carry data in both directions. Identify where this happens in the client/server exchange.

Part 2 complete: The Python TCP client and Python TCP server communicate successfully inside the Ubuntu VM, and you can identify the VM IPv4 address needed for Part 3.

Evidence: Include a screenshot showing the Python server receiving your message and the Python client receiving the server acknowledgment. Also record the VM IPv4 address you intend to use in Part 3.

Part 3: ESP32-S3 TCP Client to the Same Python Server

You will now replace the Python client from Part 2 with an ESP32-S3 client written in C using ESP-IDF. The Python server remains the same. This is the central exercise in the lab: two different computing platforms, operating environments, and programming languages communicate because both implement the same TCP/IP and application-level exchange.

1. Network topology for Part 3

The intended topology is:

```
ESP32-S3  -- Wi-Fi / TCP -->  Ubuntu VM running tcp_server.py
                same local network
```

For direct ESP32-S3-to-VM communication, the ESP32-S3 must be able to route packets to the VM. In Lab 1, NAT networking was recommended because it is convenient for outbound Internet access. For this part, bridged networking is usually preferable because it gives the VM its own address on the same local network as the laptop and ESP32-S3.

Managed Wi-Fi warning: Some campus, enterprise, hotel, and guest networks isolate wireless clients from one another. If the ESP32-S3 and VM cannot communicate even though both have Internet access, use a phone hotspot or course-recommended local network whose client-to-client communication is permitted. If your hotspot has the option of “maximize compatibility”, please toggle that option. This puts the hotspot on 2.4 GHz which ESP32 needs to connect, otherwise it may default to a 5 GHz band.

2. Put the VM on a reachable network

Shut down the VM before changing its virtual network mode. Configure its network adapter for bridged networking when supported by your virtualization software, then restart Ubuntu.

Platform	Typical action
VirtualBox	VM Settings > Network > Adapter 1 > Attached to: Bridged Adapter. Select the laptop interface that is actually connected to the network.
UTM	Use a bridged network mode/interface when available for your VM configuration. If your current UTM configuration does not expose a bridged option that works with your host network, contact course staff before proceeding.

Bridged networking caveat: Some combinations of host Wi-Fi hardware, drivers, and virtualization software and WiFi network configurations do not support reliable bridged networking. Try a phone hotspot or another course-recommended local network. If bridged mode still does not work, contact course staff for a platform-specific fallback. One possible fallback is to keep the VM in NAT mode and configure TCP port forwarding from the laptop host to the VM; in that case, the ESP32-S3 connects to the laptop host's local-network IP address and forwarded port rather than directly to the VM's private NAT address.

After booting the VM, find its address again:

```
hostname -I
ip addr
```

Write down the IPv4 address associated with the active interface. You will configure the ESP32-S3 client to connect to this address.

3. Verify that the server is listening on a reachable address

Start the same tcp_server.py from Part 2. The supplied server should bind in a way that accepts connections arriving on the VM network interface, not only from localhost. The server output should show the port on which it is listening.

```
cd ~/ee250/lab2
python3 tcp_server.py 5000
```

In another VM terminal, you can inspect listening TCP sockets with:

```
ss -ltn
```

Look for the chosen port (for example 5000). If the server is bound to 0.0.0.0:5000, it is listening on all IPv4 interfaces. If it is bound only to 127.0.0.1:5000, remote devices will not be able to connect.

4. Prepare the ESP32-S3 starter project

On the laptop native operating system, open an ESP-IDF terminal/environment as in Lab 1. Create or copy the course-provided Lab 2 ESP32 project into your ESP-IDF workspace. The starter project will reuse the Wi-Fi connection logic from Lab 1 and add a TCP client task.

The project will contain placeholders/TODOs for the TCP-client operations. You will configure at least:

- Your 2.4 GHz Wi-Fi SSID and password, following the same credential-safety rules as Lab 1.
- The Ubuntu VM IPv4 address found above.
- The TCP server port used by tcp_server.py.
- The application message to send to the server.

When first creating/copying the project, set the target and build:

```
idf.py set-target esp32s3
idf.py build
```

5. Understand the ESP32-S3 client sequence

The ESP32-S3 client should perform the same logical application steps as tcp_client.py from Part 2:

Python client in Part 2	**ESP32-S3 C client in Part 3**
Create a TCP socket	Create a TCP socket
Specify server IP address and port	Specify the same server IP address and port
connect()	connect()
sendall()/send()	send()
recv()	recv()
close()	close()

ESP-IDF provides a BSD-sockets-compatible interface through its networking stack. The syntax and data structures differ from Python, but the underlying client sequence is the same. Proceed and fill in the TODOs of esp32_tcp_client.c

Application protocol: TCP provides a reliable byte stream, but your applications must still agree on what the bytes mean. The ESP32-S3 client must send the same kind of application message expected by tcp_server.py and understand the server acknowledgment.

6. Build, flash, and monitor the ESP32-S3 client

With the project configured, build and flash it using the serial port for your board:

```
idf.py build
idf.py -p PORT flash monitor
```

Replace PORT with your ESP32-S3 serial port if automatic detection does not work.

A successful serial-monitor run should show, in order:

- The ESP32-S3 associates with Wi-Fi.
- The ESP32-S3 receives its own IPv4 address by DHCP.
- The client identifies the VM server address and port.
- The TCP connection succeeds.
- The ESP32-S3 sends its application message.
- The ESP32-S3 receives and prints the same acknowledgment that tcp_client.py received in Part 2.

At the same time, the unchanged Python server in the VM should print the message it received from the ESP32-S3.

7. Demonstrate that the server is unchanged

Do not modify tcp_server.py between the successful Part 2 Python-client test and the Part 3 ESP32-S3 test, except for any course-directed configuration that applies equally to both tests. The point is to show that the same server can communicate with two different client implementations.

Run the experiments back-to-back if convenient:

1. Start tcp_server.py in the VM.
2. Connect with tcp_client.py and complete one message/acknowledgment exchange.
3. Restart the server if the starter server handles only one connection at a time.
4. Connect with the ESP32-S3 and complete the same message/acknowledgment exchange.

Your screenshots should make this comparison easy to see.

Questions for Part 3

1. Did tcp_server.py need to know that the second client was an ESP32-S3 written in C rather than a Python program? Why or why not?
2. Why can the ESP32-S3 not use 127.0.0.1 to reach the server running in the Ubuntu VM?
3. Why can a VM in NAT mode have Internet access yet still be difficult for the ESP32-S3 to contact directly?
4. What role does the TCP port number play when both the ESP32-S3 and VM already have IP addresses?
5. What does TCP guarantee for the byte stream, and what does the application still have to define for itself?

Part 3 complete: The ESP32-S3 connects over Wi-Fi/TCP to the same Python server used in Part 2, sends an application message, and receives the server acknowledgment.

Evidence: Include screenshots showing (a) the VM server receiving a message from the ESP32-S3 and (b) the ESP32-S3 serial monitor showing a successful connection, sent message, and received acknowledgment.

Optional Challenge

For intrinsically motivated students who want to feel like they really got the hang of this, and don't need the external validation of a grade 🙂!

Implement a simple **UDP or TCP server on the ESP32-S3** that turns the onboard LED on or off when it receives an appropriate message from a corresponding UDP/TCP client running on the Linux VM. For example, your client could send `LED ON` and `LED OFF` commands.

Note that this **reverses the roles used earlier in the lab**: the ESP32-S3 is now the **server**, listening for incoming connections or messages, while the Linux VM acts as the **client**.

Troubleshooting

Q: The tc command says "Operation not permitted" or is missing.
A: Make sure you are running the command with sudo and that iproute2 is installed. Run sudo apt install -y iproute2.

Q: I forgot to remove the 50% loss rule.
A: Run sudo tc qdisc del dev lo root netem. If no rule exists, tc may report an error; that is harmless.

Q: The Python client cannot connect to the Python server on localhost.
A: Confirm that the server is running, both programs use the same TCP port, and no previous server process is still occupying the port.

Q: The server reports "Address already in use."
A: A previous process may still be listening on the port. Use ss -ltnp to inspect listening sockets, stop the old process, or use the course-approved alternate port.

Q: The ESP32-S3 gets Wi-Fi and an IP address but connect() fails.
A: Check the VM IP address, server port, VM network mode, server bind address, and whether the Wi-Fi network permits client-to-client communication.

Q: The ESP32-S3 can reach the network but still cannot connect to the TCP server running in the VM.

A: First verify that the ESP32-S3 and VM are on reachable networks, that you are using the VM's correct IP address and server port, and that tcp_server.py is listening on a reachable address. If Ubuntu's firewall is enabled, it may also be necessary to allow the TCP port used by the lab. Check with sudo ufw status; if it is active, allow only the lab port, for example sudo ufw allow 5000/tcp. Do not disable the firewall broadly just to make the lab work.

Q: The VM has Internet access in NAT mode, but the ESP32-S3 cannot connect to it.
A: This is expected on many NAT configurations. For Part 3, use bridged networking so the VM has a locally reachable address, unless course staff provides a platform-specific alternative.

Q: The VM and ESP32-S3 appear to have very different private IP ranges.

A: They may not be on the same reachable local network. Recheck the VM network mode and confirm that the laptop and ESP32-S3 are using the same access point/hotspot. If your hotspot has the option of "maximize compatibility", please toggle that option. This puts the hotspot on 2.4 GHz which ESP32 needs to connect, otherwise it may default to a 5 GHz band.

Q: The server is listening only on 127.0.0.1.

A: A remote ESP32-S3 cannot reach a loopback-only listener. Use the supplied server configuration that binds to all required interfaces (typically 0.0.0.0) or the VM network address.

Q: The ESP32-S3 connects but the server/client text looks truncated or combined.

A: Remember that TCP is a byte stream rather than a message-preserving protocol. Follow the framing/termination behavior defined by the supplied starter code and do not assume that one send() always corresponds to exactly one recv().

Q: idf.py is not found.

A: Activate the ESP-IDF environment exactly as in Lab 1. On macOS/Linux this may require . $HOME/esp/esp-idf/export.sh (or your equivalent installation path); on Windows use the ESP-IDF terminal/environment installed in Lab 1.

Submission

Each student should submit via Brightspace one PDF file containing the required screenshots and written responses, together with the requested source files separately if instructed by course staff. The final source-code packaging requirements will be specified when the starter code is released.

Your PDF should contain:

- Part 1 evidence: TCP and UDP netcat communication and at least one packet-loss experiment.
- Part 2 evidence: the Python TCP server receiving a message and the Python client receiving the acknowledgment; the VM IPv4 address used for Part 3.
- Part 3 evidence: the unchanged Python server receiving the ESP32-S3 message and the ESP32-S3 serial monitor showing successful Wi-Fi, TCP connection, sent message, and received acknowledgment.
- Answers to all questions in Parts 1-3.
- An explicit acknowledgment of all assistance received, including AI tools, people, and websites, with a brief description of what help was provided.

Credential safety: Before submission, inspect every screenshot and source file. Remove or obscure real Wi-Fi passwords, private keys, API keys, and other credentials.

Rubric

Points	Check	Evidence
6 (Demo)	Part 1: TCP vs. UDP	Correct TCP/UDP tests, loss experiment.
6 (Demo)	Part 2: Python sockets	Python client/server exchange works; student understands address, port, and client/server socket roles.

Points	Check	Evidence
8 (Demo)	Part 3: ESP32-S3 to VM	ESP32-S3 communicates with the same Python server and demonstrates send/receive over TCP.
10	Submission	Complete evidence, code, answers, readable screenshots, and acknowledgment of assistance; no exposed credentials.
70	Interview	Answer questions posed by the TA/CP about this lab to their satisfaction, demonstrating that you really understood each part, and did not just do them mechanically.
100	Total	

Optional learning resources

- Python socket programming documentation: https://docs.python.org/3/library/socket.html
- Linux ss command: https://man7.org/linux/man-pages/man8/ss.8.html
- Linux traffic control / netem documentation: https://man7.org/linux/man-pages/man8/tc-netem.8.html
- ESP-IDF BSD Sockets API: https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/lwip.html
- ESP-IDF TCP client/server examples: use the examples distributed with the same ESP-IDF version used in the course.